

0.64. Quatre formes ensemble

Le 0.63, plus un aller-retour : le D va et vient en bas à droite. Quatre lettres, quatre formes.

1. Ce qu'il faut comprendre

C'est le 0.63, plus une ligne : un aller-retour (le 0.60) pour le D, en bas à droite.

Ce qui est nouveau ici : la quatrième forme. `ALPHABET[3]` est la 4e lettre, le **D** : on compte à partir de 0 (A = 0, B = 1, C = 2, D = 3).

Change un nombre et relance : chaque forme a sa taille, son sens et sa vitesse. Tout ce qu'on a vu depuis le 0.34 est là.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <losange>` → [0.56.3. La fonction losange\(\) — tourner en losange](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <rectangle>` → [0.57.3. La fonction rectangle\(\) — tourner en rectangle](#)
- `#include <spirale>` → [0.58.3. La fonction spirale\(\) — tourner en spirale](#)
- `#include <aller_retour>` → [0.59.3. La fonction aller retour\(\) — aller et revenir](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2819 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      49 o ← la console, à l'allumage - la mise en route de la console
losange()      449 o ← main() - une fonction fournie par le langage
rectangle()    527 o ← main() - une fonction fournie par le langage
spirale()      442 o ← main() - une fonction fournie par le langage
aller_retour() 400 o ← main() - une fonction fournie par le langage
main()         130 o ← démarrage - ta fonction
VBlank         8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage  28 o ← image() - attend l'image suivante
AttendreVBlank 16 o ← démarrage - attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases - attend de pouvoir écrire une case
AdresseCase    10 o ← poser(), effacer() - trouve l'adresse d'une case de l'écran
EffacerCases   12 o ← effacer() - efface des cases
CopierTuiles   18 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage - vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles - la police entière
```

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

```
poser()        x 14  182 o sur place, dans losange(), rectangle(), spirale(), aller_retour() + AdresseCase (10 o)
effacer()       x 10  150 o sur place, dans losange(), rectangle(), spirale(), aller_retour() + AdresseCase (10 o), EffacerCases (12 o)
```

```

    image()          x 11    33 o sur place, dans losange(), rectangle(), spirale(), aller_retour(), main() +
AttendreImage (28 o)
Tes fonctions appelées :
    rectangle()      x 1     35 o sur place, dans main()
    losange()        x 1     31 o sur place, dans main()
    aller_retour() x 1     30 o sur place, dans main()
    spirale()        x 1     27 o sur place, dans main()

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

#include <losange>          // une tuile qui tourne en losange
#include <ALPHABET>         // les lettres de la police : ALPHABET[0] est le A
#include <rectangle>        // une tuile qui tourne en rectangle
#include <spirale>          // une tuile qui tourne en spirale
#include <aller_retour>     // une tuile qui va et revient

```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() {
    //   colonnes 2 à 6           colonnes 10 à 18
    //   +-----+             +-----+
    //   |  A : losange  |       |  B : rectangle  |   lignes 2 à 6
    //   +-----+             +-----+
    //   +-----+             +-----+
    //   |  C : spirale  |       |  D : aller-retour |   lignes 10 à 14
    //   +-----+             +-----+

    losange(4, 4, ALPHABET[0], 2, 1, 100, 1);          // 1. le A
    rectangle(14, 4, ALPHABET[1], 4, 2, -1, 100, 1);    // 2. le B
    spirale(4, 12, ALPHABET[2], 2, 1, 100);             // 3. le C (le 0.63)

    // 4. La ligne ajoutée : le D (ALPHABET[3], la 4e lettre), un aller-retour
    //   de 7 cases vers la droite, 100 ms par pas, 1 fois.
    aller_retour(10, 13, ALPHABET[3], 7, 0, 100, 1);

    while (true) {
        image();          // les quatre lettres sont revenues à leur place
    }
}

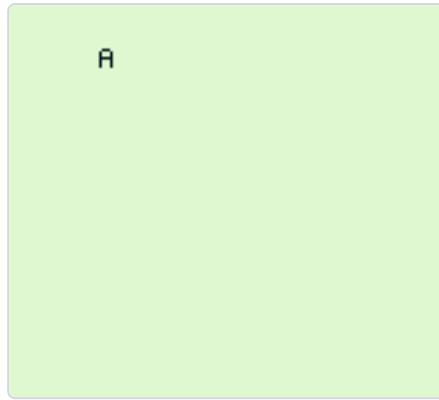
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



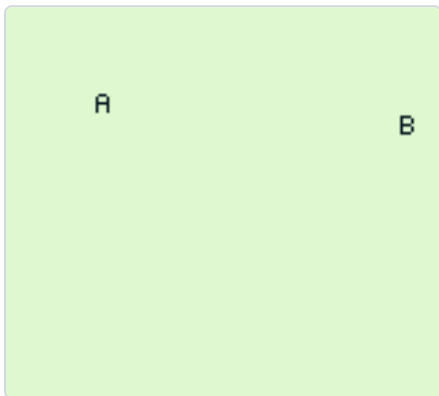
la toute première image — la console vient de démarrer — image 1



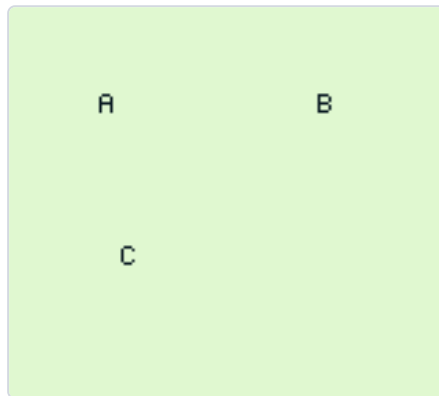
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce qu'on doit voir

Le A fait un losange, le B un rectangle, le C une spirale, le D un aller-retour. Chacun finit à sa place.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ D : l'aller-retour jusqu'à (17, 13)
- ✓ chacun à sa place : A (4, 4), B (14, 4), C (4, 12), D (10, 13) (4,4 14,4 4,12 10,13)

Ensuite : **0.64.1. Quatre formes ensemble — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++